

Algorithms for the Multidimensional Knapsack Problem (MKP)

Problem Definition & Complexity Analysis

Presented By

Shovon Roy – 2005010

Tusher Bhomik – 2005046

Md. Rabby Hossain – 2005053

Souvik Mandol – 2005069

Sagor Chanda – 2005071

Iffat Bin Hossain – 2005087

January 26, 2026

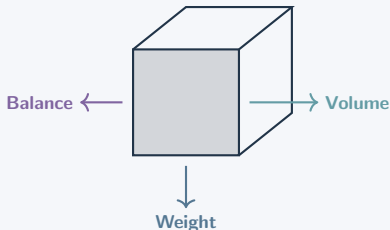
Problem Definition

Problem Definition: The "Cargo Plane" Analogy

The Intuition

Imagine you are loading a cargo plane. Unlike a standard knapsack (where you only care about weight), here you have **multiple limits** that must ALL be satisfied simultaneously:

- **Limit 1 (W):** Max Takeoff Weight.
- **Limit 2 (V):** Cargo Bay Volume.
- **Limit 3 (B):** Center of Gravity / Balance.



Formal Math

Maximize Profit $Z = \sum p_j x_j$ subject to:

$$\sum_{j=1}^n w_{ij} x_j \leq C_i$$

For all $i = 1 \dots m$

(where i represents each dimension like Weight, Volume, etc.)

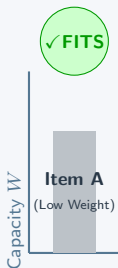
Visual Comparison: Why is MKP Harder?

The "Hidden" Constraint Problem

In Standard Knapsack, we only check one dimension. In MKP, an item might look "good" in one dimension but "break" another.

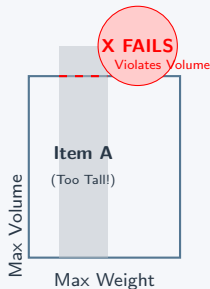
1. Standard Knapsack

(Only Weight matters)



2. Multidimensional

(Weight + Volume)



The Math Unpacked: A System of Constraints

Notation Guide:

Symbol	Name	Meaning in Analogy
n	Items	Total cargo boxes available.
m	Dimensions	The number of "gates" (Weight, Vol, etc.).
x_j	Decision	1 if we load item j , 0 if we leave it.
w_{ij}	Consumption	How much "Weight" item j uses.
C_i	Capacity	The limit of the plane for dimension i .

The "Coupled" Constraints

The difficulty comes because every single item x_j appears in **all** m **equations** simultaneously:

$$\text{Maximize } Z = \sum_{j=1}^n p_j x_j$$

Subject to:

$$\text{Dim 1 (Weight): } w_{1,1}x_1 + w_{1,2}x_2 + \cdots + w_{1,n}x_n \leq C_1$$

$$\text{Dim 2 (Volume): } w_{2,1}x_1 + w_{2,2}x_2 + \cdots + w_{2,n}x_n \leq C_2$$

...

$$\text{Dim } m \text{ (Other): } w_{m,1}x_1 + w_{m,2}x_2 + \cdots + w_{m,n}x_n \leq C_m$$

Formal Problem Statement

Definition

The Multidimensional Knapsack Problem (MKP) is a combinatorial optimization problem where we must select a subset of n items to maximize total profit, such that the total consumption of resources does not exceed the capacity limits for any of the m available dimensions.

Formulation

Given a set of items $N = \{1, \dots, n\}$ and dimensions $M = \{1, \dots, m\}$:

$$\text{Maximize } z = \sum_{j=1}^n p_j x_j$$

$$\text{Subject to } \sum_{j=1}^n w_{ij} x_j \leq C_i, \quad \forall i \in M$$

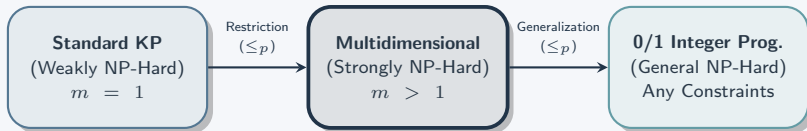
Where $x_j \in \{0, 1\}$ is the binary decision variable.

Complexity & Reductions

Complexity Analysis: The "Sandwich" Hierarchy

Where does MKP fit?

To understand the hardness of MKP, we locate it between two well-known problems using **Polynomial-Time Reductions**.



Reduction Direction	Implication for Us
From Knapsack (Left)	Proves MKP is NP-Hard (Inherited difficulty).
To ILP (Right)	Proves MKP is Solvable by general solvers.

Reduction 1: From Standard Knapsack ($KP \leq_p MKP$)

The Logic: "Restriction"

Think of MKP as a "**Universal Tool**" that handles m constraints. Standard Knapsack (KP) is just a specific job where $m = 1$.

"If I build a machine that can lift ANY object (MKP), it can certainly lift a simple box (KP)."

The Mathematical Proof:

1. **Given:** An instance of Standard Knapsack (Items n , Weights w_j , Capacity C).
2. **Construct:** An MKP instance with dimensions $m = 1$.
3. **Map:** Set the MKP capacity $C_1 = C$ and weights $w_{1j} = w_j$.

Conclusion: Inheritance of Hardness

Since Standard Knapsack is known to be **NP-Hard**, and it is just a "special case" embedded inside our problem, **MKP must be at least as hard.**

Reduction 2: Generalization to ILP ($MKP \leq_p ILP$)

The "Universal Parent" Definition

Integer Linear Programming (ILP) can solve **any** problem that satisfies three specific conditions. We map our problem to these three criteria:

1. General ILP Requirement

A. Linear Objective Function

(Must be a straight line sum)

→ $\text{Max } \sum p_j x_j$
(Profit is linear)

B. Linear Constraints

(Equalities or Inequalities)

→ $\sum w_{ij} x_j \leq C_i$
(Capacity limits are linear)

C. Integer Decision Variables

(Whole numbers only)

→ $x_j \in \{0, 1\}$
(Binary is a specific integer)

Conclusion

Since MKP satisfies all 3 requirements perfectly, it is legally an ILP problem. Therefore, standard ILP solvers (CPLEX, Gurobi) can accept MKP input without modification.

The Matrix Transformation

The Translation Step

To use an ILP solver (like CPLEX), we must translate our specific w_{ij} and C_i values into the standard Matrix Format: $\mathbf{Ax} \leq \mathbf{b}$.

Constructing the Matrices:

$$\underbrace{\begin{bmatrix} w_{1,1} & w_{1,2} & \dots & w_{1,n} \\ w_{2,1} & w_{2,2} & \dots & w_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m,1} & w_{m,2} & \dots & w_{m,n} \end{bmatrix}}_{\substack{\mathbf{A} \text{ (Constraint Matrix)} \\ \text{Size: } m \times n}} \times \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}}_{\substack{\mathbf{x} \text{ (Decision Vector)} \\ \text{Size: } n \times 1}} \leq \underbrace{\begin{bmatrix} C_1 \\ C_2 \\ \vdots \\ C_m \end{bmatrix}}_{\substack{\mathbf{b} \text{ (Capacity Vector)} \\ \text{Size: } m \times 1}}$$

Solvability Proof

Since we successfully mapped every MKP variable to a valid ILP matrix component, we have proven that **MKP is solvable** by any standard Branch-and-Cut algorithm used in commercial software.

Existing Algorithms

Exact Exponential-Time Algorithms

Exact algorithms guarantee finding the optimal solution but generally have exponential time complexity.

Algorithm	Description
Branch-and-Bound (B&B)	Explores the solution space as a tree. Uses LP-relaxation to calculate upper bounds. Prunes branches where the bound is lower than the current best.
Dynamic Programming (DP)	Extends the classic 0/1 knapsack DP to m dimensions. Requires a table of size $O(C_1 \cdot C_2 \dots C_m)$. Only feasible for very small constraints ($m \leq 2$).
Integer Linear Programming (ILP)	Solves the problem using solvers (e.g., CPLEX) with advanced cut-generation (cover cuts, clique cuts) to tighten the formulation.

Approximation Algorithms

Approximation algorithms trade optimality for theoretical performance guarantees.

Algorithm	Description
LP-Rounding	Solves the Linear Programming relaxation ($x_j \in [0, 1]$). Variables $x_j \geq 0.5$ are rounded to 1 (others to 0), followed by a repair phase to restore feasibility.
PTAS (Frieze & Clarke)	A Polynomial Time Approximation Scheme for fixed m . Enumerates all subsets of size k , then fills the remaining capacity greedily.

Randomized Algorithms

These algorithms use probabilistic choices to escape local optima or simplify complex decisions.

Algorithm	Description
Randomized Greedy	Instead of always choosing the best item, it selects randomly from a candidate list of the top k items. .
Random Walk	Iteratively moves from the current solution to a random neighbor (by flipping a bit). Can accept worse solutions with low probability to traverse the solution landscape.

Heuristic Algorithms

Constructive heuristics build a solution step-by-step using greedy logic.
Very fast but no guarantees to optimal solution.

Algorithm	Description
Toyoda's Method (Primal Greedy)	The classic heuristic for MKP. It builds a solution step-by-step, always picking the item with the highest profit-to-resource-usage ratio.
Senju-Toyoda (Dual Greedy)	Starts with all items selected (infeasible). Iteratively drops items with the lowest profit-to-resource-usage ratio until all constraints are satisfied.
Pivot & Complement	Enhances greedy solutions by attempting to swap a selected item with one or more non-selected items to utilize remaining unused capacity.

Metaheuristic Algorithms

Metaheuristics explore the solution space using strategies inspired by nature to escape local optima.

Algorithm	Description
Genetic Algorithm (Chu & Beasley)	Uses binary tournament selection and a specialized "repair operator" to fix infeasible offspring generated during crossover, ensuring population feasibility.
Tabu Search	Local search that maintains a "Tabu list" of recently visited states to avoid cycles. Uses oscillating strategies to explore feasible and infeasible regions.
Ant Colony Optimization (ACO)	Constructs solutions probabilistically based on pheromone trails. Pheromones are updated based on the quality of the knapsack solutions found.

Complexity Comparison Summary

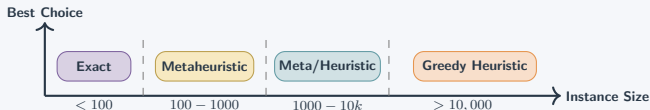
When to Use Each Category?

The choice depends on instance size (n), constraint tightness (m), and time constraints.

Category	Complexity (Typical)	Limit (n items)	Use Case
Exact	$O(2^n)$ or $O(mC^m)$	< 100	Optimality Required
Approximation	Polynomial $O(n^k)$	$> 10,000$	Theoretical Bounds
Randomized	$O(S \cdot mn)$	Any	Diversity / Baseline
Heuristic	$O(mn)$ to $O(mn^2)$	$> 50,000$	Real-time Systems
Metaheuristic	$O(I \cdot P \cdot mn)$	$1,000 - 10,000$	High-Quality / Offline

Notation: n : Items, m : Dimensions, I : Iterations, P : Population, S : Samples, C : Capacity.

Best Choice Selection:



Implementation of Algorithms

Implementation Strategy: Overview

Selection Logic

We will implement one **Constructive Heuristic** (for speed/baseline) and one **Randomized Algorithm** (for diversity/comparison).

1. Toyota's Method

Type: Constructive Greedy Heuristic

- **Category:** Heuristic (Slide 4)
- **Core Idea:** Selects items based on a dynamic "bang-for-buck" ratio.
- **Why?** The standard deterministic baseline. Fast and consistent.

2. Randomized Greedy

Type: Randomized Constructive

- **Category:** Randomized (Slide 3)
- **Core Idea:** Selects randomly from the top k items at each step.
- **Why?** Extremely easy extension of Toyota. Runs multiple times to find better solutions.

Design: Toyota's Heuristic

Goal

Construct a feasible solution from scratch by picking the "best" item step-by-step.

Algorithm Steps:

1. **Input:** Set of items N , Constraints m , Capacities C_i .
2. Start with empty knapsack $x = \{0, 0, \dots\}$.
3. **Calculate Gradient:** For every unselected item j , calculate ratio:

$$\text{Ratio}_j = \frac{p_j}{\sum_{i=1}^m \frac{w_{ij}}{\text{RemainingCap}_i}}$$

4. **Select:** Pick item j with the **highest Ratio**.
5. **Check:** If item j fits in all dimensions, add it ($x_j \leftarrow 1$). Update remaining capacities.
6. **Repeat** until no more items fit.

Design: Randomized Greedy

Goal

Introduce diversity by picking from the "good" items rather than just the "best" one.

Process Flow:

1. **Init:** Empty Knapsack.
2. **Candidate List (RCL):**
 - Calculate gradients/ratios exactly like Toyota's Method.
 - Instead of picking the top 1, identify the **Top** k items (e.g., top 5).
3. **Selection:**
 - Pick **randomly** from these Top k candidates.
4. **Update:** Add item, update capacities, repeat until full.
5. **Iteration (Key Step):**
 - Run this whole process N times (e.g., 100 times).
 - Return the **best** solution found across all runs.

Comparison of Selected Algorithms

Property	Toyoda's Method	Randomized Greedy
Category	Heuristic (Constructive)	Randomized
Logic	Deterministic	Probabilistic
Execution	Single Pass	Multi-Pass (N iterations)
Complexity	$O(m \cdot n^2)$	$O(N \cdot m \cdot n^2)$
Solution Quality	Good Baseline	Better (Local Optima Escape)
Implementation	Very Easy	Very Easy (Wrapper)
Use Case	Real-time Systems	Offline Optimization

Conclusion

We choose **Toyoda's Method** as a fast baseline. We implement **Randomized Greedy** to demonstrate how simple randomization can improve upon the deterministic greedy approach.

Research Questions

Goal

To analyze the trade-offs between mathematical intelligence and probabilistic exploration.

- **RQ 1: Precision vs. Exploration**

In tight environments, can randomized selection outperform Toyoda's penalty-aware logic?

- **RQ 2: Scalability of Intelligence**

As the number of dimensions (m) increases, does Toyoda's Method maintain a lower optimality gap?

- **RQ 3: Reliability in Production**

How many Randomized Greedy runs are needed to match one deterministic Toyoda solution?

- **RQ 4: Topology Sensitivity**

How does knapsack tightness affect the variance and stability of Randomized Greedy?

Experimentation and Evaluation Design

Benchmark Selection & Test Scenarios

Goal

To define a standardized testing environment that challenges both algorithms fairly.

- **Standard Benchmark Library**
OR-Library (Chu & Beasley instances) for reproducible evaluation.
- **Problem Scale**
 - Small: $n = 100$, $m = 5$
 - Medium: $n = 250$, $m = 10$
 - Large: $n = 500$, $m = 30$
- **Resource Tightness (α)**
 - Tight: $\alpha = 0.25$
 - Loose: $\alpha = 0.75$

Comparative Metrics

Goal

To provide quantitative evidence of which algorithm performs better under specific conditions.

- **1. Optimality Gap**

Percentage difference from the known optimal value:

$$Gap = \frac{Profit_{Optimal} - Profit_{Algorithm}}{Profit_{Optimal}} \times 100\%$$

- **2. Computation Time (Latency)**

CPU time in milliseconds. Both methods use similar penalty-based ratio computations per iteration, but Randomized Greedy incurs higher total time due to repeated runs.

- **3. Best-of-N Performance (Randomized Greedy)**

- 20 runs per instance
- Record Best Solution
- Measure Coefficient of Variation (stability of results)

Theoretical and Real-World Applications

Why MKP Matters: Theoretical Significance

Bridge Between Simple and General Problems

The Multidimensional Knapsack Problem (MKP) plays a central role in theoretical computer science and operations research because it lies between simple knapsack models and full general integer programming systems.

- **A Testbed for General Integer Programming (ILP)**
MKP is the simplest non-trivial form of 0/1 Integer Linear Programming. Unlike general ILP, it uses only $\leq$ (capacity) constraints and positive coefficients.
- Because of this simplicity, MKP is the **gold standard** for:
 - Testing commercial solvers (e.g., CPLEX, Gurobi)
 - Developing cutting-plane and branch-and-bound techniques

Why One Constraint Is Not Enough

In real systems, decisions rarely depend on a single resource. MKP models situations where multiple scarce resources must be managed simultaneously.

- **General Interpretation**

Each item consumes several resources at once, and selecting an item may:

- Help satisfy one constraint
- Violate another constraint

- This makes MKP a natural model for:

- Resource allocation
- Capacity planning
- Multi-criteria decision making

1. Logistics & Transportation

Cargo Loading Problem

- Scenario: Loading aircraft or shipping containers
- Dimensions:
 - Weight limit
 - Volume limit
 - Balance / center of gravity

An item may be light but too bulky, blocking other profitable cargo.

2. Capital Budgeting

Project Selection in Finance

- Scenario: Selecting R&D and investment projects
- Dimensions:
 - Budget in Year 1
 - Budget in Year 2
 - ... Budget in Year m

Goal: Maximize total profit without exceeding any yearly budget.

3. Cloud Computing

Resource Allocation in Data Centers

- Scenario: Assigning VMs to physical servers
- Dimensions:
 - CPU cores
 - RAM
 - Disk space
 - Network bandwidth

Cloud providers solve massive MKPs to maximize revenue per server.

4. Multi-Attribute Auctions

Digital Advertising & Procurement

- Scenario: Selecting winning bids
- Dimensions:
 - Total budget
 - Frequency limits
 - Demographic reach

The platform selects bids that maximize revenue under all constraints.

References

- Chu & Beasley (1998)
[Semantic Scholar Link](#)
- Kellerer, Pferschy, Pisinger (Survey)
[SCIRP Survey Link](#)
- Google OR-Tools (Knapsack Solver)
[OR-Tools Documentation](#)
- Frieze & Clarke (1984)
[HAL / INRIA Paper](#)

Thank You!

Questions and Discussion

Algorithms for the Multidimensional Knapsack Problem (MKP)
Problem Definition & Complexity Analysis